

AUDIT — APPLICATION DE GESTION DE STOCK

Audit technique, fonctionnel, métier, UX/UI, sécurité et architecture
Vision cible et feuille de route d'évolution

Application	gestion-stock (« GestStock ») — magasin d'optique multi-dépôts
Dépôt analysé	branche <code>main</code> , commit <code>3945251</code> (« revu data et logique »)
Date de l'audit	3 octobre 2026
Périmètre	257 fichiers suivis, ~18 800 lignes (TS 12 624, HTML 3 381, SCSS 2 837)
Méthode	Lecture intégrale du code, build de production, exécution des tests, <code>npm audit</code> . Aucun fichier du projet n'a été modifié.

MATURITÉ ACTUELLE

31 / 100

Prototype fonctionnel avancé — non exploitable en production en l'état

Sommaire

1. Résumé exécutif
2. Présentation de l'application
3. Architecture actuelle
4. Cartographie des modules
5. Audit fonctionnel (métier)
6. Audit technique
7. Audit base de données
8. Audit API
9. Audit UX/UI
10. Audit sécurité
11. Audit performance
12. Audit tests
13. Audit qualité du code
14. Fonctionnalités manquantes
15. Recommandations
16. Architecture cible
17. Modèle métier cible
18. Workflows recommandés
19. Matrice Existant / Cible
20. Score de maturité
21. Registre des risques
22. Roadmap
23. Priorisation Impact / Effort
24. Conclusion
25. Annexes (conventions, preuves, méthode)

Conventions de statut : **EXISTE** vérifié dans le code · **PARTIEL** commencé mais incomplet · **ABSENT** non trouvé · **PROBLÉMATIQUE** présent mais défaillant · **À CONFIRMER** invérifiable avec les éléments disponibles.

Abréviation utilisée dans les preuves : **MBI** = `src/app/core/interceptors/mock-backend.interceptor.ts` (le « backend simulé », 1 976 lignes). Les numéros de ligne sont ceux du commit `3945251`.

1. Résumé exécutif

Ce qu'est réellement l'application. GestStock est une **application Angular 21 sans backend réel**. Toute la logique métier (stock, ventes, achats, caisse, crédit client, permissions) est exécutée **dans le navigateur**, par un intercepteur HTTP qui simule une API (`MBI` , 69 routes). Les données sont **en mémoire** : elles sont perdues à chaque rechargement de page et ne sont pas partagées entre postes. En production (`environment.prod.ts`), les mocks sont désactivés et l'application pointe vers `https://api.example.com/api` , **qui n'existe pas** : l'application de production est donc non fonctionnelle.

Ce qui est solide. Le périmètre fonctionnel couvert par l'interface est large et cohérent pour un commerce d'optique : produits, stock multi-dépôts, ventes détail/gros avec crédit client, commandes fournisseurs (réception, facture, paiements partiels), caisse (ouverture, opérations, clôture avec écart), inventaire, dépenses, rapports, journal d'audit, utilisateurs. L'architecture front (composants standalone, lazy loading, pattern *Feature* → *Facade* → *ApiService*) est propre et réutilisable. Le contrat d'API est déjà esquissé, et des schémas PostgreSQL/MongoDB existent (`src/app/core/mocks/seeds/` , `DATABASE_SCHEMA.md`).

Ce qui bloque.

1. **Aucun backend, aucune persistance** : c'est le chantier n°1, préalable à tout le reste.
2. **Sécurité inexistante dans les faits** : aucune route n'est protégée par un guard. En mode mock, toutes les requêtes s'exécutent avec les droits **admin**, car l'intercepteur JWT est placé après le mock. Le mot de passe est `admin` pour tous les comptes.
3. **Bug de stock critique** : la réception d'une commande fournisseur **n'augmente pas** le stock réellement utilisé (stock par dépôt). Le stock affiché et celui contrôlé à la vente divergent dès la première réception.
4. **Intégrité métier fragile** : prix et prix d'achat d'une vente envoyés par le client (marge falsifiable). Stock modifiable directement depuis la fiche produit, sans mouvement. Mouvements, ventes et inventaires sans dépôt rattaché. Ni annulation ni retour.
5. **Traçabilité de façade** : l'écran d'audit est riche, mais seules 2 actions sur 69 écrivent réellement dans le journal. Les 50 entrées visibles sont des données de démonstration.
6. **Tests quasi inexistants** : 2 tests, dont 1 en échec.

Verdict. L'existant est une **excellente maquette fonctionnelle** et une bonne base front. Il **n'est pas une application de gestion exploitable**. Une réécriture n'est pas nécessaire : il faut **construire le backend** (en transposant les règles déjà codées dans `MBI`), **brancher le front dessus** et **corriger le modèle de stock**. Effort estimé pour une V1 exploitable (phases 0 à 2) : **12 à 17 semaines-développeur**.

Prochaine étape concrète : valider la stack backend (recommandation : NestJS + PostgreSQL + Prisma, cohérent avec TypeScript et avec `sql-seed.sql`), puis réaliser en 2 semaines le « socle » (auth réelle, produits, stock par dépôt avec mouvements, transactions).

2. Présentation de l'application

Élément	Constat	Preuve
Domaine	Magasin d'optique (montures, verres, lentilles, accessoires, solutions) en Côte d'Ivoire, montants en FCFA	<code>src/app/core/mocks/mock-db.ts:29-35</code> , données clients/fournisseurs
Type	SPA Angular, interface en français	<code>package.json</code> , templates
Utilisateurs cibles	Admin, Gestionnaire, Caissier	<code>src/app/core/models/role.model.ts</code>
Multi-dépôts	2 dépôts de démonstration (<code>wh_1</code> Magasin principal, <code>wh_2</code> Dépôt secondaire), sélecteur dans l'en-tête	<code>layout-shell.component.html</code> , <code>warehouse.interceptor.ts</code>
Backend	Simulé dans le navigateur (intercepteur HTTP)	MBI:388-392
Persistance	Mémoire uniquement (variables <code>let mock...</code>), réinitialisée au rechargement	MBI:49-329
Production	<code>useMocks: false</code> + <code>apiUrl: https://api.example.com/api</code> (factice)	<code>src/environments/environment.prod.ts</code>
Historique Git	8 commits du 26/04/2026 au 12/05/2026	<code>git log</code>
Documentation	<code>README.md</code> (générique Angular CLI), <code>TECHNICAL_AUDIT.md</code> (audit antérieur, mai 2026), <code>DATABASE_SCHEMA.md</code> (schéma MongoDB <i>reconstruit</i>)	racine du dépôt

Note sur l'audit antérieur (`TECHNICAL_AUDIT.md`). Il a été relu mais **pas pris pour acquis**. Chaque point a été revérifié. Plusieurs bugs qu'il signale sont **toujours présents** (B1, B3, B5, B6, B8, B10, B11, B12, B14, A1, A4, A5). D'autres ont évolué : `transactionsCount` est désormais renseigné (MBI:1865, mais non filtré par date), et `isAuthenticated()` vérifie maintenant l'expiration (`auth/services/auth.service.ts:77-86`), sans qu'aucun guard ne l'utilise.

3. Architecture actuelle

3.1 Stack technique

Couche	Technologie	Version installée	Remarque
Framework	Angular (standalone, control flow <code>@if/@for</code>)	21.2.10	Récent ; vulnérabilités HIGH signalées par <code>npm audit</code> pour < 21.2.19
UI	Angular Material + CDK	21.2.8	Tables, paginator, sort, drawers, dialogs
État	RxJS <code>BehaviorSubject</code> + <code>combineLatest</code> (ViewModel observable)	rxjs 7.8	Pas de store global ; signals marginaux
Graphiques	Chart.js	4.4.x	Dashboard, rapports
Exports	xlsx (SheetJS) 0.18.5, jsPDF 4.2.1	—	xlsx : vulnérabilités HIGH sans correctif npm
Tests	Vitest + jsdom via <code>@angular/build:unit-test</code>	vitest 4	1 fichier de test
Backend	Aucun — intercepteur <code>mockBackendInterceptor</code>	—	69 routes simulées
Base de données	Aucune — schémas proposés non branchés	—	<code>sql-seed.sql</code> (20 tables, 11 index), <code>mongo-seed.js</code>

3.2 Schéma d'architecture actuel



3.3 Patterns et stratégies

Sujet	Stratégie actuelle	Évaluation
Séparation front/back	Contrat HTTP propre, mais le « back » vit dans le front	PROBLÉMATIQUE
Organisation	<code>core/</code> (models, services, guards, interceptors, mocks) + <code>features/<module>/{data,pages,ui}</code>	EXISTE bonne structure
Gestion d'état	Facade par feature, <code>vm\$</code> combiné	EXISTE cohérent mais verbeux
Validation	Reactive Forms côté UI + contrôles ad hoc dans MBI	PARTIEL pas de schéma partagé
Authentification	JWT simulé non signé (<code>jwt.sim.ts</code>), tokens en <code>localStorage</code> / <code>sessionStorage</code>	PROBLÉMATIQUE
Autorisation	<code>hasRole()</code> dans MBI ; guards écrits mais jamais branchés	PROBLÉMATIQUE
Contexte dépôt	En-tête <code>X-Warehouse-Id</code> injecté sur toutes les requêtes	EXISTE bonne idée, mal exploitée côté données
Erreurs	<code>apiErrorInterceptor</code> (snackbar global) + <code>toApiError</code> dupliqué dans 11 services	PARTIEL
Persistance	Aucune	ABSENT

4. Cartographie des modules

Module	Route	Existence	État	Complétude	Observations (preuves)
Authentification	/login, /forgot-password	EXISTE	PROBLÉMATIQUE	35 %	Login fonctionnel en mock ; mot de passe <code>admin</code> universel (MBI:485) ; « mot de passe oublié » simulé ; aucun guard sur les routes (<code>layout.routes.ts</code>)
Dashboard	/dashboard	EXISTE	PARTIEL	40 %	3 KPI (CA du jour, marge du jour, unités en stock) ; courbe « évolution du stock » fiction (<code>dashboard.facade.ts:152-165</code>)
Produits	/products	EXISTE	PARTIEL	55 %	CRUD + recherche/tri/pagination client ; stock éditable dans la fiche (contourne les mouvements)
Catégories	—	PARTIEL	Lecture seule	15 %	Seul GET <code>/categories</code> existe (MBI:632) ; 5 catégories figées, pas d'écran
Stock / mouvements	/stock	EXISTE	PROBLÉMATIQUE	45 %	Historique + saisie manuelle ; mouvements sans dépôt ; aucun contrôle de rôle (MBI:1610)
Dépôts / transferts	/warehouses, /warehouses/transfer	EXISTE	PARTIEL	40 %	Création + transfert immédiat mono-produit ; pas de modification/suppression ni d'emplacements
Ventes (POS)	/sales	EXISTE	PROBLÉMATIQUE	50 %	Panier, détail/gros, crédit, facture PDF ; prix fournis par le client ; ni annulation, ni retour, ni remise, ni TVA
Clients	/clients, /clients/:id	EXISTE	PARTIEL	60 %	CRUD, plafond de crédit, paiements, historique ; surpaiement possible
Achats	/purchases	EXISTE	PROBLÉMATIQUE	50 %	Commande → réception → facture → paiement ; réception sans effet sur le stock réel (MBI:1219-1229)
Fournisseurs	/suppliers, /suppliers/:id	EXISTE	PARTIEL	60 %	CRUD, historique, paiements ; suppression laisse des commandes orphelines (MBI:916-917)
Caisse	/cash-register/{open,close,history}	EXISTE	PARTIEL	55 %	Session unique globale (tous postes, tous dépôts) ; ventes non rattachées à la session
Dépenses	/expenses	EXISTE	PARTIEL	50 %	Création/suppression ; pas de modification ; catégorie en texte libre
Inventaire	/inventory/{count,history}	EXISTE	PROBLÉMATIQUE	35 %	Application immédiate, sans brouillon ni validation ; quantités pré-remplies = théorique ; sessions sans dépôt
Rapports	/reports	EXISTE	PARTIEL	40 %	Jour/mois/année calculés côté client (corrects) ; endpoints <code>/reports/*</code> faux mais inutilisés ; export Excel
Rendez-vous (optique)	/appointments	EXISTE	PARTIEL	50 %	CRUD ; aucun contrôle de rôle ; endpoint statut bogué (MBI:1768) mais non appelé par l'UI

Module	Route	Existence	État	Complétude	Observations (preuves)
Journal d'audit	/audit-logs	EXISTE	PROBLÉMATIQUE	25 %	Belle UI (filtres, stats, export CSV) ; 2 actions réellement tracées ; 50 entrées de démonstration
Utilisateurs	/users	EXISTE	PARTIEL	45 %	CRUD + activation ; pas de mot de passe ; un utilisateur désactivé peut se connecter
Notifications / alertes	—	PARTIEL	Indicateur visuel	10 %	Uniquement le libellé « Stock faible / Rupture » dans la liste produits
Paramètres (société, TVA, numérotation)	—	ABSENT	—	0 %	—
Retours clients / fournisseurs	—	ABSENT	—	0 %	—
Lots / péremption	—	ABSENT	—	0 %	—
Code-barres / QR	—	ABSENT	—	0 %	—
Code mort	ReportsPageComponent , SalesPageComponent , StockPageComponent , UsersPageComponent , query.engine , network.sim , mock-factory , mock-db-large , core/guards/* , CoreModule , SharedModule	PROBLÉMATIQUE	—	—	Jamais référencés par le routing ni par d'autres modules

5. Audit fonctionnel (métier)

5.A Produits

Attribut / fonction	Statut	Preuve / commentaire
Création / modification	EXISTE	<code>product-drawer.component.ts</code> , MBI:638-713
Suppression	PROBLÉMATIQUE	Suppression physique, sans contrôle des ventes/mouvements liés, ni audit (MBI:715-729)
Référence / SKU	PARTIEL	Requis côté UI ; côté API auto-généré <code>SKU-XXXX</code> aléatoire, sans contrôle d'unicité (MBI:649)
Code-barres (EAN/GTIN)	ABSENT	Champ inexistant dans <code>product.model.ts</code>
Nom	PARTIEL	Validé côté UI ; l'API accepte un nom vide (défaut « Nouveau produit », MBI:650)
Description	ABSENT	—
Catégorie	EXISTE	<code>categoryId</code> ; défaut silencieux <code>cat_1</code> côté API
Marque	ABSENT	Présente seulement dans le libellé (« Ray-Ban... ») — critique en optique
Unité / conditionnement	ABSENT	Pas de notion de boîte de lentilles, de paire, etc.
Prix d'achat	PROBLÉMATIQUE	Écrasé par le dernier prix reçu (MBI:1227) ; pas de CUMP
Prix de vente détail / gros	EXISTE	<code>retailPrice</code> , <code>wholesalePrice</code>
Marge	PARTIEL	Calculée à la vente uniquement ; aucun contrôle « prix de vente < prix d'achat »
TVA	ABSENT	Aucun champ ni calcul
Seuil minimum	EXISTE	<code>alertThreshold</code>
Seuil maximum / surstock	ABSENT	—
Stock actuel	PROBLÉMATIQUE	Champ <code>stockQuantity</code> dénormalisé et stock par dépôt : deux sources de vérité (voir 5.C)
Stock réservé / disponible	ABSENT	—
Image	ABSENT	—
Statut actif / inactif	ABSENT	Seule la suppression physique existe
Variantes (couleur, taille, puissance)	ABSENT	Indispensable pour montures et lentilles (dioptrie, rayon, diamètre)
Historique des prix	ABSENT	Seul l'audit d'un <code>UPDATE</code> produit conserve <code>before/after</code> (MBI:701)

5.B Catégories

PARTIEL — Seule la lecture est possible (`GET /categories`, MBI:632-636). Il n'y a ni création, ni modification, ni suppression, ni hiérarchie, ni contrôle d'unicité, ni activation. Le modèle se limite à `{id, name}` (`category.model.ts`). **Risque** : impossible d'ajouter une gamme sans modifier le code.

5.C Stock — analyse approfondie

Mécanisme réel. Le stock est **stocké directement** dans une table `Record<warehouseId, Record<productId, qty>>` (MBI:309-327), modifiée par `setStock / adjustStock`. Des mouvements sont **en plus** journalisés, mais le stock **n'est jamais recalculé** à partir d'eux. Parallèlement, `Product.stockQuantity` existe comme champ propre. Il est mis à jour par la réception (seulement), puis **écrasé à l'affichage** par la valeur du dépôt courant (MBI:530).

Opération	Statut	Effet sur stock dépôt	Mouvement créé	Problème
Stock initial	PARTIEL	Oui (dépôt courant)	Non	Pas de mouvement « stock initial » → historique incomplet (MBI:665-668)

Opération	Statut	Effet sur stock dépôt	Mouvement créé	Problème
Entrée par réception fournisseur	PROBLÉMATIQUE	Non	Oui (SUPPLY)	Bug critique : seul <code>product.stockQuantity</code> est incrémenté (MBI:1224-1229), valeur ignorée ensuite. Le mouvement existe, mais pas le stock.
Sortie par vente	EXISTE	Oui	Oui (SALE)	Mouvement sans <code>warehouseId</code> (dépôt seulement dans la note)
Mouvement manuel (entrée, perte, ajustement)	PARTIEL	Oui	Oui	Aucun contrôle de rôle ; motif <code>LOSS / ADJUSTMENT</code> sans justification obligatoire (MBI:1610-1648)
Modification via fiche produit	PROBLÉMATIQUE	Oui (<code>setStock</code>)	Non	Contournement total de la traçabilité (MBI:692-695, champ <code>stockQuantity</code> du formulaire)
Transfert inter-dépôts	PARTIEL	Oui (2 dépôts)	Oui (2 × <code>ADJUSTMENT</code>)	Motif <code>TRANSFER</code> absent ; immédiat (pas de « en transit ») ; mono-produit
Inventaire	PARTIEL	Oui (<code>setStock</code>)	Oui (<code>ADJUSTMENT</code>)	Voir 5.H
Stock réservé / théorique / disponible	ABSENT	—	—	—
Utilisateur responsable	PROBLÉMATIQUE	—	<code>createdByUserId</code>	En mock, toujours <code>u_admin</code> (voir sécurité S2)
Date / heure	EXISTE	—	<code>createdAtISO</code>	—

Risques d'incohérence identifiés.

1. Divergence **stock affiché / stock vendable** après chaque réception (critique).
2. Σ mouvements $\neq$ stock (stock initial et modifications par fiche sans mouvement) : **aucune réconciliation possible**.
3. Mouvements non rattachés à un dépôt : impossible de reconstituer le stock d'un dépôt ou de filtrer l'historique par dépôt.
4. Aucune atomicité : en backend réel, une vente multi-lignes pourrait décrémenter une partie du stock puis échouer.
5. Concurrence non gérée (deux caisses vendant le dernier article).
6. Suppression d'un produit : ventes et mouvements orphelins (le nom n'est plus résolu, `stock.facade.ts:112`).

5.D Entrepôts / magasins

Capacité	Statut	Commentaire
Plusieurs dépôts	EXISTE	Création par nom (MBI:553-575) ; modèle <code>{id, name}</code> seulement
Stock par dépôt	EXISTE	Mais vue globale « tous dépôts » absente
Transfert entre dépôts	PARTIEL	Immédiat, sans bon de transfert, validation ni réception
Emplacements, zones, rayons	ABSENT	—
Restriction utilisateur $\leftrightarrow$ dépôt	ABSENT	<code>User.magasin</code> est un libellé libre jamais utilisé pour filtrer ; le sélecteur d'en-tête est libre
Modifier / désactiver un dépôt	ABSENT	—
Ventes, caisse, inventaires par dépôt	PROBLÉMATIQUE	<code>Sale</code> , <code>InventorySession</code> , <code>CashRegisterSession</code> n'ont pas de <code>warehouseId</code>

Intérêt d'aller plus loin : pour un opticien à 2 sites, les emplacements fins (rayons) sont **P3**. En revanche, le rattachement systématique au dépôt de chaque document (vente, mouvement, inventaire, caisse) et la vue consolidée sont **P0/P1**.

5.E Achats

Processus cible : *Fournisseur* $\rightarrow$ *commande* $\rightarrow$ *réception* $\rightarrow$ *contrôle* $\rightarrow$ *stock* $\rightarrow$ *facture* $\rightarrow$ *paiement*.

Étape	Statut	Commentaire
Fournisseurs	EXISTE	CRUD, délai de livraison
Commande d'achat + lignes	EXISTE	Statuts PENDING → DELIVERED uniquement
Modification / annulation de commande	ABSENT	Pas de statut CANCELLED, ni de brouillon
Réception complète	PROBLÉMATIQUE	Sans effet sur le stock dépôt (bug critique) ; dépôt de réception non choisi
Réception partielle	ABSENT	Tout ou rien
Contrôle qualité / écarts à réception	ABSENT	—
Prix d'achat / historique	PROBLÉMATIQUE	Dernier prix écrase l'ancien ; pas d'historique ni de CUMP
Facture fournisseur	PARTIEL	Numéro + date ; montant = total commande (pas d'écart facture/commande, ni de TVA)
Paielements partiels	EXISTE	Contrôle « ≤ reste à payer »
Dettes fournisseurs	PARTIEL	Calculables par commande ; pas de balance âgée ni d'échéances
Retours fournisseurs / avoirs	ABSENT	—
Commande déclenchée par seuil	ABSENT	—

5.F Ventes

Fonction	Statut	Commentaire
Clients	EXISTE	Nom, téléphone, email, plafond de crédit
Vente comptoir (panier → paiement)	EXISTE	sales.facade.ts, MBI:1657-1753
Détail / gros	EXISTE	Re-tarification du panier au changement de type
Vente à crédit + plafond	PARTIEL	Plafond contrôlé ; crédit sans client accepté par l'API (UI seule bloque, sales-shell-page.component.ts:181)
Modes de paiement	PARTIEL	CASH, MOBILE_MONEY, BANK_TRANSFER ; pas de paiement mixte
Prix de vente	PROBLÉMATIQUE	unitPrice et purchasePrice viennent du client (MBI:1663-1676) : prix et marge falsifiables
Facture	PARTIEL	PDF généré côté navigateur (jsPDF) ; pas de numérotation légale séquentielle, de mentions légales ni de TVA
Commande client / devis / préparation / livraison	ABSENT	Pas de cycle commande → livraison
Remise (ligne / globale)	ABSENT	—
TVA	ABSENT	—
Annulation de vente	ABSENT	Aucune route DELETE / cancel
Retours	ABSENT	—
Marge / bénéfice	PARTIEL	Calculés sur le prix d'achat envoyé par le client
Stock après vente	EXISTE	Décrément dépôt + mouvement SALE
Lien vente ↔ session de caisse	ABSENT	Une vente cash est possible caisse fermée ; rattachement par fenêtre horaire seulement (MBI:273-279)
Ordonnance / prescription (optique)	ABSENT	À considérer : verres correcteurs sur prescription

5.G Clients et crédit

PARTIEL — La dette est calculée à la volée : $\Sigma \text{ventes} - \Sigma \text{payé à la vente} - \Sigma \text{paiements}$ (MBI:1687-1691). Points faibles :

- **surpaiement accepté**, aucune comparaison avec la dette (MBI:1571) ;

- la **suppression d'un client efface ses paiements** mais conserve ses ventes, ce qui fausse les historiques (MBI:1528-1529) ;
- pas d'échéance, de relance ni de balance âgée ;
- le paiement d'une dette en espèces n'entre pas dans la caisse.

5.H Inventaire

Exigence	Statut	Commentaire
Inventaire manuel	EXISTE	Saisie des quantités physiques dans un tableau
Inventaire global	PROBLÉMATIQUE	Tous les produits sont listés et pré-remplis avec la quantité théorique (<code>inventory-count-page.component.ts:61</code>). Un article non compté est réputé conforme, ce qui masque les pertes.
Inventaire partiel (catégorie, zone)	ABSENT	—
Écart théorique / réel	EXISTE	différence par ligne
Ouverture → comptage → validation	ABSENT	Une seule action applique immédiatement les ajustements (MBI:789-805)
Gel du stock / ventes pendant comptage	ABSENT	—
Justification des écarts	ABSENT	Note globale uniquement
Valorisation de l'écart	ABSENT	—
Historique / responsable / date	PARTIEL	Oui, mais dépôt non enregistré dans la session (<code>inventory.model.ts</code>)
Double comptage / validation par un tiers	ABSENT	—

5.I Retours

ABSENT — Aucun retour client (total ou partiel, motif, remboursement, avoir, remise en stock, produit défectueux), aucun retour fournisseur. Le motif de mouvement `RETURN` n'existe pas (`stock-movement.model.ts`).

5.J Alertes

Alerte	Statut
Stock faible (≤ seuil)	PARTIEL — libellé dans la liste produits, pour le dépôt courant seulement ; ni notification ni liste dédiée
Rupture	PARTIEL — libellé « Rupture »
Surstock	ABSENT
Produit expiré / proche expiration	ABSENT
Commande fournisseur en retard	ABSENT (le <code>deliveryLeadTimeDays</code> existe mais n'est pas exploité)
Paiement client en retard	ABSENT
Inventaire nécessaire	ABSENT

5.K Lots et dates d'expiration

ABSENT — Il n'y a ni numéro de lot, ni dates de fabrication ou de péremption, ni quantité par lot, ni FIFO/FEFO. **Pertinence élevée pour ce commerce** : les lentilles de contact et les solutions d'entretien sont périssables et soumises à traçabilité (rappels de lots fabricant). Ailleurs, c'est indispensable en pharmacie, agroalimentaire, cosmétique et pièces détachées réglementées. **Recommandation : P2**, limitée aux catégories « Lentilles » et « Solutions » (flag `trackBatches` par produit), avec sortie FEFO.

5.L Code-barres / QR

ABSENT — Il n'y a ni champ, ni scan, ni impression d'étiquettes. **Stratégie réaliste** :

1. ajouter `barcode` (EAN-13) sur le produit, avec index unique ;
2. au POS et à l'inventaire, utiliser une **douchette USB/Bluetooth en émulation clavier** (aucun SDK, un champ texte focalisé suffit) ;
3. générer des étiquettes internes (Code 128) pour les produits sans EAN, en PDF A4 multi-étiquettes ;
4. la caméra mobile (lib `@zxing/browser`) vient seulement en phase 5.

5.M Utilisateurs, rôles, permissions

Aspect	Statut	Commentaire
CRUD utilisateurs	EXISTE	Admin uniquement côté API
Mot de passe	ABSENT	Aucun champ ; <code>admin</code> pour tous (MBI:485)
Rôles	PROBLÉMATIQUE	Deux modèles concurrents : <code>Role = ADMIN/CAISSIER/GESTIONNAIRE</code> (<code>core/models/role.model.ts</code>) et <code>UserRole = ADMIN/MANAGER/EMPLOYEE</code> (<code>auth/models/user.model.ts</code>), ponts via <code>as unknown as any</code> (<code>core/services/auth.service.ts:40</code>)
Permissions fines (lecture/création/modif/suppression/validation)	ABSENT	Rôles codés en dur par route
Accès par module (menu)	ABSENT	Menu identique pour tous (<code>layout-shell.component.ts:62-78</code>)
Restriction par dépôt / entreprise	ABSENT	—
Désactivation effective	PROBLÉMATIQUE	<code>isActive</code> non vérifié au login (MBI:482-489)
Auto-suppression de l'admin	PROBLÉMATIQUE	Rien n'empêche de supprimer le dernier admin (MBI:1438-1449)

5.N Audit et traçabilité

PROBLÉMATIQUE . Le modèle `AuditLogEntry` est bien conçu (utilisateur, rôle, action, entité, `before/after`, `changes`, IP, statut). L'écran est complet. **Mais** :

- `appendAudit()` n'est appelé que **2 fois** : mise à jour produit (MBI:701) et création de vente (MBI:1741) ;
- les 50 entrées affichées sont **codées en dur** (MBI:70-121) ;
- l'IP vaut `127.0.0.1` et le rôle `ADMIN` par défaut ;
- le journal est plafonné à 500 entrées en mémoire ;
- il n'y a ni immuabilité, ni horodatage serveur ;
- connexions, déconnexions, suppressions, réceptions, paiements, inventaires et transferts **ne sont pas tracés**.

5.O Tableau de bord

KPI	Statut	Fiabilité
CA du jour	EXISTE	Moyenne : calcul client sur toutes les ventes, tous dépôts confondus, date UTC
Marge du jour	EXISTE	Faible : dépend du prix d'achat envoyé par le client
Unités en stock	EXISTE	Faible : dépôt courant seulement, en unités (pas en valeur), faussé par le bug de réception
Ventes 14 jours, top 8 produits	EXISTE	Correct sur les données disponibles
Évolution du stock	PROBLÉMATIQUE	Fictive : $\text{stock actuel} \times (1 - 0,02 \cdot i)$ (<code>dashboard.facade.ts:160</code>). Trompeuse, à retirer.
Valeur du stock, ruptures, produits sous seuil, achats, créances, dettes, rotation	ABSENT	—

5.P Rapports

PARTIEL. La page `/reports` filtre correctement par jour, mois ou année **côté client** (`reports.facade.ts:110-121`). Elle affiche CA, marge brute, nombre de transactions et top produits, avec export Excel. Les dépenses ne sont **pas déduites** : il n'y a pas de bénéfice net. Les endpoints `/api/reports/daily|monthly|yearly` sont **faux** (totaux non filtrés, `lossCount: 0`, MBI:1862-1970) mais **inutilisés** : le seul consommateur, `ReportsPageComponent`, n'est pas routé. Absents : état et valorisation du stock, mouvements, achats, inventaires, produits dormants, rotation, créances, dettes, historique des prix, performance par dépôt ou par vendeur.

5.Q Finance / gestion commerciale

Élément	Statut
Chiffre d'affaires, marge brute	PARTIEL
Coût d'achat (CUMP / FIFO)	ABSENT (dernier prix)
TVA, remises	ABSENT
Paiements clients / fournisseurs	EXISTE
Crédits clients, dettes fournisseurs	PARTIEL
Dépenses	EXISTE (simples)
Caisse	PARTIEL (globale, non rattachée aux ventes)
Bénéfice net (marge – dépenses)	ABSENT dans l'UI

Frontière recommandée. Restent **dans le périmètre (gestion commerciale)** : CA, marge, CUMP, TVA collectée/déductible *calculée*, remises, encaissements, créances, dettes, caisse, dépenses opérationnelles, exports. Restent **hors périmètre (comptabilité)** : plan comptable, écritures en partie double, grand livre, bilan, liasse fiscale, rapprochement bancaire. Prévoir plutôt un **export comptable** (journal des ventes, achats et encaissements au format CSV/FEC-like) vers le logiciel du cabinet comptable.

5.R Multi-entreprise / multi-magasin

Multi-magasin : l'en-tête `X-Warehouse-Id` est une bonne base. Il manque le rattachement des documents au dépôt, les droits par dépôt et la consolidation. L'évolution est **faisable** sans refonte. **Multi-entreprise** : rien n'existe (pas de `companyId`, pas d'isolation). C'est faisable si le backend est conçu **dès le départ** avec un `companyId` sur chaque table et un filtrage systématique (row-level security PostgreSQL ou middleware). Ce n'est **pas prioritaire** pour un opticien mono-société : **P3**, mais la colonne est à prévoir dès la phase 1 pour éviter une migration lourde.

6. Audit technique

Constat	Gravité	Preuve
Logique métier exécutée dans le navigateur (intercepteur de 1 976 lignes, 69 routes en <code>if</code> successifs)	CRITIQUE	MBI
Données en mémoire, perdues au rechargement, non partagées entre postes	CRITIQUE	MBI:49-329
Build de production sans backend (<code>api.example.com</code>)	CRITIQUE	<code>environment.prod.ts</code>
Données mock (noms, emails, téléphones) incluses dans le bundle de production	MOYEN	Bundle <code>main-*.js</code> contient <code>optivision.ci</code> (import statique de <code>mock-db.ts</code> par MBI)
Ordre des intercepteurs : <code>jwt</code> après <code>mockBackend</code>	ÉLEVÉ	<code>app.config.ts:21</code>
Deux <code>AuthService</code> , deux modèles <code>User</code> , deux jeux de <code>guards</code>	MOYEN	<code>auth/services</code> , <code>core/services</code> , <code>auth/guards</code> , <code>core/guards</code>
Aucune transaction ni atomicité dans les opérations multi-étapes	ÉLEVÉ (en backend réel)	Vente, réception, inventaire
Identifiants générés par <code>Date.now()</code> : collisions possibles en rafale	FAIBLE	Ex. MBI:609 et 617 (in/out même ms)
Gestion des dates en UTC via <code>toISOString().slice(0,10)</code> : décalage possible hors GMT	FAIBLE (GMT en CI)	<code>dashboard.facade.ts:102</code> , <code>reports.facade.ts:113</code>
Build OK ; avertissements CommonJS (<code>jspdf/canvg</code>)	FAIBLE	<code>ng build</code>

7. Audit base de données

Il n'y a **pas de base de données**. L'analyse porte sur les **modèles TypeScript** (le contrat de fait) et sur les **schémas proposés mais non branchés** (`sql-seed.sql` : 20 tables, 11 index, extension `pgcrypto` ; `mongo-seed.js` ; `DATABASE_SCHEMA.md` , « reconstruit depuis le code source »).

Problème structurel	Gravité	Détail
Pas de <code>warehouseId</code> sur <code>StockMovement</code> , <code>Sale</code> , <code>InventorySession</code> , <code>CashRegisterSession</code>	CRITIQUE	Traçabilité par dépôt impossible
Double source de vérité du stock (<code>Product.stockQuantity</code> vs table dépôt)	CRITIQUE	Cause directe du bug de réception
Lignes de vente sans snapshot (nom/SKU)	MOYEN	Historique illisible après suppression produit
Aucune contrainte d'unicité appliquée (SKU, nom de catégorie, n° facture)	ÉLEVÉ	Doublons possibles
Suppressions physiques en cascade incohérentes (client → paiements supprimés, ventes conservées ; fournisseur → historique supprimé, commandes conservées)	ÉLEVÉ	MBI:1528, 916
Pas de soft delete, pas de <code>updatedAt</code> / <code>updatedBy</code>	MOYEN	—
Montants en <code>number</code> JS (flottants)	MOYEN	En SQL : <code>NUMERIC(14,2)</code> ou entiers en FCFA
Énumérations incomplètes (<code>TRANSFER</code> , <code>RETURN</code> , <code>INITIAL</code> , <code>CANCELLED</code>)	MOYEN	<code>stock-movement.model.ts</code> , <code>purchase-order.model.ts</code>
Mot de passe par défaut <code>password123</code> dans le seed SQL	ÉLEVÉ si exécuté en prod	<code>sql-seed.sql:51</code>
<code>DATABASE_SCHEMA.md</code> décrit MongoDB, <code>sql-seed.sql</code> décrit PostgreSQL	FAIBLE	Choix de SGBD non tranché

Recommandation : PostgreSQL. Les données sont fortement relationnelles et transactionnelles (stock, argent), avec besoin de contraintes, de `SELECT ... FOR UPDATE` et d'agrégations de reporting. Le `sql-seed.sql` existant est un bon point de départ, à corriger selon la section 17.

8. Audit API

Contrat actuel : REST JSON, préfixe `/api`, réponses de liste `{items, total}`, erreurs `{message}` avec codes 400/401/403/404. **69 routes** simulées, **32 contrôles de rôle**.

Domaine	Endpoints existants	Problèmes
Auth	POST <code>/auth/login</code> , <code>/auth/refresh</code> , <code>/auth/forgot-password</code> , GET <code>/auth/me</code>	Pas de logout serveur ni de révocation ; refresh sans vérification d'expiration ni de signature ; énumération d'emails via forgot-password (404 si inconnu, MBI:514)
Produits	GET/POST <code>/products</code> , GET/PUT/DELETE <code>/products/:id</code>	Pas de pagination, recherche ni filtre serveur ; PUT modifie le stock ; POST sans validation
Catégories	GET <code>/categories</code>	CRUD manquant
Dépôts	GET/POST <code>/warehouses</code> , POST <code>/warehouses/transfer</code>	PUT/DELETE manquants ; transfert non idempotent
Stock	GET/POST <code>/stock/movements</code>	Pas de filtre dépôt ni de pagination ; pas de contrôle de rôle sur POST ; pas de GET <code>/stock</code> (niveaux par dépôt)
Ventes	GET/POST <code>/sales</code>	Prix fournis par le client ; pas de GET <code>/sales/:id</code> , <code>cancel</code> , <code>returns</code> ; pas de pagination
Clients	CRUD + <code>/customers/:id/sales</code> , <code>/payments</code>	Pas de GET <code>/customers/:id/balance</code> ; surpaiement
Fournisseurs	CRUD + <code>/purchases</code> , <code>/payments</code>	Suppression destructive
Achats	GET/POST <code>/purchases/orders</code> , <code>/:id</code> , <code>/:id/receive</code> , <code>/:id/invoice</code> , <code>/:id/pay</code>	Pas d'annulation ni de réception partielle (payload <code>deliveredAtIso</code> ignoré) ; deux routes de paiement (<code>/purchases/orders/:id/pay</code> et <code>/suppliers/:id/payments</code> avec <code>orderId</code>) : doublon
Caisse	<code>current</code> , <code>sessions</code> , <code>open</code> , <code>operations</code> , <code>close</code>	Session globale, non liée à l'utilisateur ni au dépôt
Inventaire	GET/POST <code>/inventory/sessions</code> , GET <code>/:id</code>	Pas de cycle ouverture/validation
Dépenses	GET/POST <code>/expenses</code> , DELETE <code>/:id</code>	Pas de PUT ni de filtres de période
Rapports	GET <code>/reports/daily monthly yearly</code>	Résultats faux (non filtrés) ; inutilisés
Utilisateurs	CRUD + PATCH <code>/:id/active</code>	Pas de gestion de mot de passe
Audit	GET <code>/audit-logs</code> , <code>/:id</code> , <code>/stats</code> , <code>/export</code>	Seul domaine paginé ; export ignore <code>search</code>
RDV	CRUD + PATCH <code>/:id/status</code>	Aucun contrôle de rôle ; PATCH <code>status</code> extrait l'id au mauvais index (<code>split('/')[4] = "status"</code>) → toujours 404 (MBI:1768)

Endpoints manquants prioritaires : GET `/stock/levels?warehouseId=`, POST `/sales/:id/cancel`, POST `/sales/:id/returns`, POST `/purchases/orders/:id/receipts` (partielle), POST `/purchases/orders/:id/cancel`, CRUD `/categories`, PUT/DELETE `/warehouses/:id`, cycle `/inventory/sessions/:id/{lines,validate}`, `/transfers` (cycle), `/reports/stock-valuation`, `/reports/receivables`, `/reports/payables`, POST `/auth/logout`, POST `/users/:id/password`.

Règles transverses à imposer côté backend : pagination `?page&size&sort&q`, validation par DTO (class-validator/Zod), prix et coûts toujours relus en base, clés d'idempotence sur les écritures financières, codes d'erreur métier stables (`STOCK_INSUFFICIENT`, `CREDIT_LIMIT_EXCEEDED` ...).

9. Audit UX/UI

Points forts.

- Interface Angular Material cohérente, avec thème global (`styles.scss` , 586 lignes).
- Navigation latérale repliable et sélecteur de magasin visible dans l'en-tête.
- Drawers latéraux pour création et détail, qui conservent le contexte de la liste.
- Dialogues de confirmation pour les suppressions (produits, clients, fournisseurs, dépenses, utilisateurs, RDV) et la réception.
- Tables avec tri et pagination (8 écrans), recherche avec debounce, snackbar d'erreur global.
- Facture PDF et export Excel.

Points faibles.

Constat	Impact	Recommandation
Menu identique pour tous les rôles ; un caissier voit « Utilisateurs », « Audit »...	Confusion, erreurs 403	Menu filtré par permission
Sidenav en <code>mode="side"</code> toujours ouverte, pas de <code>BreakpointObserver</code>	Inutilisable sur mobile/tablette (inventaire en rayon, vente tablette)	Mode <code>over</code> < 960 px
Inventaire : un tableau de tous les produits sans pagination ni recherche, pré-rempli	Ingérable au-delà de ~200 articles ; erreurs silencieuses	Comptage par scan ou recherche, lignes « non comptées » distinctes
POS : recherche produit par liste, sans scan ni raccourcis clavier	Lenteur en caisse	Champ scan focalisé, raccourcis (F2 paiement, Échap annuler)
Stock modifiable depuis la fiche produit	Erreur de manipulation non tracée	Champ en lecture seule + bouton « Ajuster » qui crée un mouvement motivé
Pas d'annulation ni de correction de vente	Les erreurs de caisse sont irréparables	Annulation motivée avec droit dédié
Graphique d'évolution du stock fictif	Décision sur de fausses données	Retirer ou baser sur les mouvements
Pas d'actions groupées ni d'export sur les listes	Productivité	Export CSV/Excel par liste, sélection multiple
Messages techniques bruts (« Articles invalides. »)	Compréhension	Messages métier contextualisés

Montée en charge des écrans (toutes les listes chargent l'intégralité des données puis filtrent dans le navigateur) :

Écran	10 produits	1 000 produits	10 000 produits	100 000 mouvements
Produits	OK	OK	Lent (chargement complet + filtre client)	—
POS (sélection produit)	OK	Difficile (liste déroulante)	Inutilisable	—
Inventaire	OK	Inutilisable (1 000 champs)	Inutilisable	—
Stock / mouvements	OK	OK	Lent	Inutilisable (tout chargé)
Dashboard / rapports	OK	OK (ventes chargées en totalité)	—	Lent à inutilisable (agrégations client sur toutes les ventes)

Solutions : pagination et recherche **serveur**, filtres avancés persistés dans l'URL, autocomplete avec scan au POS, vues compactes, export serveur, agrégations pré-calculées pour le reporting.

10. Audit sécurité

#	Problème	Gravité	Preuve	Recommandation
S1	Aucune route protégée : aucun <code>canActivate</code> / <code>canMatch</code> n'est déclaré, toute l'application est accessible sans connexion	CRITIQUE	<code>routing.ts</code> , <code>layout.routes.ts</code> , <code>features/**/*.routes.ts</code>	Brancher <code>AuthGuard</code> sur le layout + <code>permissionGuard</code> par route
S2	En mock, toutes les requêtes sont exécutées en admin : l'intercepteur JWT est après le mock, donc aucun token n'est transmis et le fallback <code>mockUsers[0]</code> s'applique. Contrôles de rôle inopérants, audit toujours au nom de <code>admin</code> .	CRITIQUE	<code>app.config.ts:21</code> , MBI:377-382	Placer <code>jwt</code> avant <code>mockBackend</code> ; supprimer le fallback
S3	Pas de backend : toute règle de sécurité côté client est contournable (console, modification du JS)	CRITIQUE	Architecture	Backend avec autorisation serveur
S4	Mot de passe universel <code>admin</code> , affiché sur l'écran de login	CRITIQUE (si déployé)	MBI:485, <code>login.component.html:129-130</code>	Hash Argon2/bcrypt, politique de mot de passe
S5	JWT non signé (signature factice) ; refresh accepté sans vérification d'expiration	ÉLEVÉ	<code>jwt.sim.ts:128</code> , MBI:497-499	JWT signé côté serveur, rotation des refresh tokens
S6	Tokens en <code>localStorage</code> / <code>sessionStorage</code> (exposés en cas de XSS)	ÉLEVÉ	<code>auth/services/auth.service.ts:40-47</code>	Refresh token en cookie <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Strict</code> , access token en mémoire
S7	Prix de vente et prix d'achat fournis par le client	ÉLEVÉ	MBI:1663-1676	Relire les prix en base ; remise soumise à droit et plafond
S8	Utilisateur désactivé peut se connecter ; pas de verrouillage après échecs	ÉLEVÉ	MBI:482-489	Contrôle <code>isActive</code> , rate limiting, verrouillage temporaire
S9	Mouvement de stock créable par tout utilisateur authentifié ; RDV sans contrôle de rôle	ÉLEVÉ	MBI:1610, 1756-1849	Permissions explicites par action
S10	Dépendances vulnérables : <code>@angular/*</code> < 21.2.19 (HIGH), <code>xlsx</code> 0.18.5 (HIGH, pas de correctif npm), <code>dompurify</code> , <code>fflate</code> (MODERATE) — 10 vulnérabilités	ÉLEVÉ	<code>npm audit --omit=dev</code>	Mettre à jour Angular ; remplacer <code>xlsx</code> npm par la distribution SheetJS CDN $\geq 0.20.2$ ou par <code>exceljs</code>
S11	Énumération des comptes via « mot de passe oublié » (404 si email inconnu)	MOYEN	MBI:514	Réponse identique dans tous les cas
S12	Données de démonstration (emails, téléphones) embarquées dans le bundle de production	MOYEN	Bundle <code>main-*.js</code>	Charger les mocks via import dynamique conditionné à l'environnement
S13	Suppression du dernier administrateur possible	MOYEN	MBI:1438-1449	Garde-fou serveur
S14	Pas de CSP, ni d'en-têtes de sécurité, ni de config CORS (pas de serveur)	MOYEN	—	À définir avec le backend (helmet, CORS restreint)
S15	Seed SQL avec mot de passe par défaut <code>password123</code>	MOYEN	<code>sql-seed.sql:51</code>	Seed de dev uniquement, forcer le changement au 1er login
S16	Pas d'upload de fichiers aujourd'hui	—	—	À sécuriser lors de l'ajout des images (type MIME, taille, stockage objet)

11. Audit performance

Risque	Gravité	Détail
Listes non paginées côté serveur (produits, ventes, mouvements, clients, commandes)	ÉLEVÉ	Tout est chargé puis filtré par <code>MatTableDataSource</code>
Dashboard et rapports agrègent toutes les ventes dans le navigateur	ÉLEVÉ	<code>dashboard.facade.ts:82</code> , <code>reports.facade.ts:42-45</code>
Recherches linéaires <code>find()</code> dans des boucles (enrichissement des mouvements, $O(n \times m)$)	MOYEN	<code>stock.facade.ts:111-112</code>
Double appel API au rafraîchissement de l'inventaire	FAIBLE	<code>inventory.facade.ts:41-45</code>
Souscriptions sans désabonnement dans des constructeurs	FAIBLE	<code>inventory-count-page.component.ts:58</code>
<code>OnPush</code> sur 16 composants sur 51	FAIBLE	—
Bundle initial raisonnable (lazy loading par feature) ; <code>xlsx/jspdf</code> chargés à la demande	OK	<code>reports-shell-page.component.ts:95</code> , <code>invoice-drawer.component.ts:31</code>
Index base de données	À CONCEVOIR	Prévoir <code>(warehouse_id, product_id)</code> unique sur les niveaux de stock ; <code>(product_id, created_at)</code> et <code>(warehouse_id, created_at)</code> sur les mouvements ; <code>(created_at)</code> , <code>(customer_id)</code> sur les ventes ; <code>sku</code> et <code>barcode</code> uniques

12. Audit tests

Type	Existant	Résultat
Unitaires	<code>src/app/app.spec.ts</code> (2 tests générés par le CLI)	1 échec : attend un <code><h1>Hello, gestion-stock</h1></code> qui n'existe plus
Intégration / API	Aucun	—
Frontend (composants, facades)	Aucun	—
E2E	Aucun (pas de framework installé)	—

Zones critiques sans tests : calcul et contrôle du stock (vente, réception, transfert, inventaire), crédit client, paiement fournisseur (reste à payer), réconciliation de caisse, permissions.

Stratégie proposée :

1. tests unitaires des **services de domaine backend** (règles de stock et d'argent), objectif $\geq 80\%$ sur ces modules ;
2. tests d'**intégration API** sur base PostgreSQL éphémère (Testcontainers), incluant la concurrence (deux ventes simultanées du dernier article) ;
3. tests de facades front (Vitest, déjà configuré) ;
4. **5 à 8 parcours E2E Playwright** : connexion, vente cash, vente à crédit, réception, inventaire, clôture de caisse ;
5. exécution en CI à chaque PR.

13. Audit qualité du code

Constat	Mesure	Gravité
Intercepteur monolithique : 69 handlers en <code>if</code> successifs, logique métier + données + RBAC mélangées	1 976 lignes	ÉLEVÉ (dette, mais destiné à être remplacé)
Mutations typées <code>unknown</code> , puis castées <code>as any</code> à l'usage	27 méthodes ; 33 <code>any</code>	MOYEN
Fonction <code>toApiError</code> copiée dans 11 services	11 copies	FAIBLE
Doublons d'authentification (2 services, 2 modèles, 2 guards)	—	MOYEN
Code mort (4 pages non routées, moteur de mock non utilisé, <code>CoreModule</code> / <code>SharedModule</code> vides)	~1 000 lignes	FAIBLE
Mutation directe d'objet partagé (RDV)	MBI:1772, 1786	FAIBLE
Conventions homogènes, TypeScript <code>strict</code> , templates stricts, nommage clair (FR métier / EN technique)	—	Point fort
Pattern Facade/VM régulier et lisible d'une feature à l'autre	—	Point fort

Pas de refonte front justifiée. La structure des features est saine. La dette se concentre dans `MBI` (à remplacer par le backend), dans le double système d'auth et dans le typage des facades.

14. Fonctionnalités manquantes

Complexité : S (≤ 3 j), M (≤ 2 sem.), L (≤ 1 mois), XL (> 1 mois).

#	Fonctionnalité	Priorité	Justification (pourquoi / problème résolu / impact métier)	Complexité	Dépendances
1	Backend API + base PostgreSQL	P0	Sans persistance ni serveur, aucune donnée n'est conservée ni partagée : l'outil est inutilisable	XL	—
2	Authentification réelle (hash, JWT signé, refresh HttpOnly, guards)	P0	Accès libre et droits admin pour tous aujourd'hui	M	1
3	Modèle de stock unique par dépôt + mouvements obligatoires et transactionnels	P0	Corrige le bug de réception et rend le stock auditable	L	1
4	Prix et coûts calculés côté serveur	P0	Empêche la falsification du CA et des marges	S	1
5	Rattachement au dépôt de toutes les pièces (vente, mouvement, inventaire, caisse)	P0	Traçabilité et reporting par magasin	M	3
6	Journal d'audit réel, systématique, immuable	P0	Lutte contre la fraude et litiges	M	1
7	Permissions RBAC (rôle → permissions) + menu filtré	P1	Séparation des tâches caissier/gestionnaire/admin	M	2
8	Annulation de vente et retours clients (avoir, remise en stock)	P1	Situations quotidiennes en boutique, impossibles aujourd'hui	M	3, 6
9	Réception partielle et annulation de commande fournisseur	P1	Les livraisons partielles sont la norme	M	3
10	CUMP (coût unitaire moyen pondéré) + historique des prix	P1	Marge exacte, valorisation du stock	M	3
11	Inventaire en workflow (ouverture → comptage → validation)	P1	Fiabilité du stock, contrôle des écarts	M	3, 5
12	Caisse par poste/utilisateur/dépôt, liée aux ventes et encaissements	P1	Réconciliation fiable, responsabilité des caissiers	M	5
13	TVA et remises (avec plafond par rôle)	P1	Factures conformes, gestion commerciale	M	4
14	CRUD catégories + marques + statut actif produit	P1	Autonomie de gestion du catalogue	S	1
15	Pagination, recherche et filtres serveur	P1	Tenue à la volumétrie	M	1
16	Code-barres (champ + scan POS/inventaire + étiquettes)	P2	Vitesse en caisse, fiabilité inventaire	M	14
17	Alertes (seuil, rupture, retard fournisseur, créance échue)	P2	Anticipation des ruptures et impayés	M	3, 12
18	Tableau de bord fiable + rapports (valorisation, rotation, dormants, créances, dettes)	P2	Pilotage	L	3, 10
19	Transferts en workflow (demande → expédition → réception)	P2	Gestion du stock en transit	M	5
20	Lots et péremption (lentilles, solutions), FEFO	P2	Rappels fabricant, pertes sur périmés	L	3
21	Variante produit (couleur, taille, dioptrie)	P2	Catalogue optique réaliste	L	14
22	Fiche patient / ordonnance liée à la vente (optique)	P2	Cœur de métier opticien	M	8

#	Fonctionnalité	Priorité	Justification (pourquoi / problème résolu / impact métier)	Complexité	Dépendances
23	Export comptable (ventes, achats, encaissements)	P2	Lien avec le cabinet comptable	S	13
24	Proposition de réapprovisionnement automatique	P3	Gain de temps achats	M	17, 18
25	Multi-entreprise (isolation <code>companyId</code>)	P3	Commercialisation SaaS éventuelle	L	1 (à prévoir dès le schéma)
26	Emplacements fins (zones, rayons)	P3	Utile en entrepôt, peu en boutique	M	5
27	PWA / mode hors-ligne POS, scan caméra mobile	P3	Coupures réseau, mobilité	L	15, 16
28	Notifications (email/SMS/WhatsApp)	P3	Relances clients, alertes	M	17

15. Recommandations

Chaque recommandation répond à : *Pourquoi ? Quel problème ? Impact métier ? Complexité ? Dépendance ? Priorité ?*

R1 — Construire le backend en transposant MBI, pas en repartant de zéro. *Pourquoi* : les 69 routes et leurs règles sont un cahier des charges exécutable. *Problème résolu* : absence de persistance et de sécurité. *Impact* : l'outil devient exploitable. *Complexité* : XL. *Dépendance* : aucune. *Priorité* : P0. Garder le mock comme environnement de démo, après avoir corrigé l'ordre des intercepteurs.

R2 — Un seul modèle de stock. La table `stock_levels(warehouse_id, product_id, quantity)` est mise à jour **uniquement** par un service `StockService.applyMovement()` transactionnel, qui écrit toujours un `stock_movement` (avec `warehouse_id`, motif, document source, utilisateur, coût unitaire). Supprimer `Product.stockQuantity` du modèle persistant. *Problème* : bug de réception et divergences. *Impact* : confiance dans le stock. *Complexité* : L. *Dépendance* : R1. P0.

R3 — Le serveur fait foi pour les prix. La vente envoie `productId`, `quantity` et éventuellement `discount`. Le serveur relit prix, coût (CUMP) et TVA. *Problème* : fraude et marges fausses. *Complexité* : S. P0.

R4 — Sécurité de base. Guards sur toutes les routes, RBAC serveur, mots de passe hachés, refresh token en cookie HttpOnly, contrôle `isActive`, rate limiting du login, mise à jour d'Angular et remplacement de `xlxs`. *Complexité* : M. P0.

R5 — Audit systématique par intercepteur ou middleware serveur sur toute écriture, plus événements métier (vente, réception, paiement, inventaire, connexion). Table en ajout seul. *Complexité* : M. P0.

R6 — Pièces commerciales à états. Vente : DRAFT → CONFIRMED → CANCELLED/RETURNED. Commande : DRAFT → ORDERED → PARTIALLY_RECEIVED → RECEIVED → CLOSED/CANCELLED. Inventaire : OPEN → COUNTING → VALIDATED. Pas de suppression physique des pièces : annulation motivée. *Complexité* : M à L. P1.

R7 — Nettoyage front ciblé (sans refonte) : un seul `AuthService`, un seul modèle `User`, type `Observable<T>` des facades, `toApiError` mutualisé, suppression du code mort, menu par permission, sidenav responsive. *Complexité* : S à M. P1.

R8 — Retirer les indicateurs trompeurs (évolution de stock fictive) et corriger les libellés (« marge brute » au lieu de « bénéfice »). *Complexité* : S. P0 (quick win).

16. Architecture cible

Principe : **conserver le front Angular** (structure, features, facades, Material) et **ajouter un backend modulaire** qui reprend le contrat d'API existant, enrichi.



Brique	Choix recommandé	Justification
Backend	NestJS (TypeScript)	Même langage que le front, modèles partageables, structure modulaire proche des features Angular
ORM / migrations	Prisma (ou TypeORM)	Migrations versionnées, typage
Base	PostgreSQL 16	Transactions, contraintes, <code>NUMERIC</code> , verrous, vues matérialisées pour le reporting
Auth	JWT d'accès 15 min + refresh rotatif en cookie <code>HttpOnly</code> ; <code>Argon2id</code>	Standard, résistant au vol de token XSS
Permissions	RBAC rôle → permissions (<code>stock.adjust</code> , <code>sales.cancel</code> ...) + portée dépôt	Souplesse sans multiplier les rôles
Fichiers	Stockage S3-compatible (MinIO en local)	Images produits, factures PDF archivées
Rapports	Requêtes SQL agrégées + vues matérialisées rafraîchies par job ; PDF de facture générés côté serveur	Exactitude, numérotation légale, performance
Audit	Table <code>audit_logs</code> en ajout seul, écrite dans la même transaction que l'action	Non-répudiation
Tâches planifiées	Alertes de seuil, retards fournisseurs, créances échues, sauvegardes	Automatisation
Intégrations	Export comptable CSV ; Mobile Money (P3) ; SMS/WhatsApp (P3)	Selon besoin
Déploiement	Docker Compose (api, db, minio) ; CI GitHub Actions (lint, tests, build)	Simple à exploiter pour une PME

17. Modèle métier cible

Entité	Statut	Commentaire / changements
User	EXISTANTE → À MODIFIER	+ passwordHash, lastLoginAt, failedAttempts ; un seul modèle
Role	EXISTANTE (type) → À MODIFIER	Deviens une table, liée à des permissions
Permission	À CRÉER	Codes module.action
UserWarehouseAccess	À CRÉER	Portée par dépôt
Company	À CRÉER (colonne seulement)	companyId prévu, sans UI multi-société (P3)
Warehouse (magasin/dépôt)	EXISTANTE → À MODIFIER	+ code, type (boutique/dépôt), address, isActive
Location (emplacement)	NON NÉCESSAIRE à court terme	P3
Product	EXISTANTE → À MODIFIER	+ barcode, brandId, unit, vatRate, isActive, trackBatches, maxThreshold, description, imageUrl, avgCost ; - stockQuantity
ProductVariant	À CRÉER (P2)	Couleur/taille/dioptrie
Category	EXISTANTE → À MODIFIER	+ parentId, isActive, unicité du nom
Brand	À CRÉER	Clé en optique
StockLevel	PARTIELLE (map mock) → À CRÉER	(warehouseId, productId) unique, quantity, reserved
StockMovement	EXISTANTE → À MODIFIER	+ warehouseId, unitCost, sourceType/sourceId, motifs INITIAL/TRANSFER_IN/TRANSFER_OUT/RETURN/INVENTORY
StockTransfer (+ lignes)	À CRÉER	Workflow avec état « en transit »
Inventory / InventoryLine	EXISTANTES → À MODIFIER	+ warehouseId, status, countedBy, validatedBy, justification par ligne
ProductBatch	À CRÉER (P2)	Lot, dates de fabrication et de péremption, quantité par dépôt
Supplier	EXISTANTE	+ isActive, conditions de paiement
PurchaseOrder / PurchaseOrderLine	EXISTANTES → À MODIFIER	Statuts étendus, warehouseId, receivedQty par ligne
GoodsReceipt (+ lignes)	À CRÉER	Réceptions partielles multiples
SupplierInvoice	PARTIELLE (objet imbriqué) → À MODIFIER	Entité propre, montants HT/TVA/TTC, échéance
Customer	EXISTANTE → À MODIFIER	+ adresse, type (particulier/institution), isActive
Patient / Prescription	À CRÉER (P2, optique)	Lien client ↔ ordonnance ↔ vente
Sale / SaleItem	EXISTANTES → À MODIFIER	+ number séquentiel, warehouseId, cashSessionId, status, remises, TVA, snapshot produit
Return (+ lignes)	À CRÉER	Client et fournisseur, motif, destination (stock/rebut)
Payment	PARTIELLE (2 modèles) → À MODIFIER	Unifier CustomerPayment / SupplierPayment ou garder deux tables cohérentes ; + mode et session de caisse
Invoice / CreditNote	À CRÉER	Facture et avoir numérotés, générés côté serveur
CashRegisterSession / CashOperation	EXISTANTES → À MODIFIER	+ warehouseId, registerId, opérateur
Expense	EXISTANTE → À MODIFIER	Catégories référencées, warehouseId, modification

Entité	Statut	Commentaire / changements
Appointment	EXISTANTE	Lier au Customer (aujourd'hui nom libre)
Notification / Alert	À CRÉER (P2)	—
AuditLog	EXISTANTE (modèle)	Modèle conservé ; alimentation à généraliser
Setting (société, TVA, numérotation)	À CRÉER	—

18. Workflows recommandés

Achat : Fournisseur → Commande (brouillon) → Commande envoyée → Réception(s) partielle(s) au dépôt choisi (contrôle des quantités et écarts) → Mouvements SUPPLY + recalcul CUMP → Facture fournisseur (rapprochement commande/réception/facture) → Paiement(s) → Clôture

Vente comptoir : Caisse ouverte (poste, dépôt) → Panier (scan) → Prix et TVA serveur, remise selon droit → Paiement (mixte possible) ou crédit si client et plafond OK → Sortie de stock transactionnelle → Facture numérotée → Rattachement à la session de caisse

Vente sur commande (optique) : Client + ordonnance → Devis → Commande (acompte) → Commande fournisseur des verres → Réception → Montage → Retrait par le client → Solde → Facture

Retour client : Vente d'origine → Sélection des lignes et quantités → Motif → Destination (remise en stock / rebut / retour fournisseur) → Avoir ou remboursement (caisse) → Mouvement RETURN → Audit

Inventaire : Ouverture (dépôt, périmètre, gel optionnel) → Comptage (scan, lignes non comptées distinctes) → Comparaison et valorisation des écarts → Justification des écarts significatifs → Validation par un responsable → Ajustements INVENTORY → Historique

Transfert : Demande (dépôt B) → Validation → Expédition depuis A (TRANSFER_OUT, stock « en transit ») → Réception en B (TRANSFER_IN, écarts signalés) → Clôture

Caisse : Ouverture (fond, opérateur, poste) → Ventes et encaissements de créances rattachés → Entrées et sorties motivées → Comptage → Écart → Validation de l'écart au-delà d'un seuil par un responsable → Clôture

19. Matrice Existant / Cible

Domaine	Existant	Problème	Cible	Priorité
Backend & données	Mock en mémoire dans le navigateur	Rien n'est persistant ni partagé	API NestJS + PostgreSQL	P0
Authentification	JWT simulé, mot de passe <code>admin</code> , routes non protégées	Accès libre	Auth réelle + guards	P0
Permissions	3 rôles codés en dur, inopérants en mock	Tout le monde est admin	RBAC permissions + portée dépôt	P0/P1
Stock	Map par dépôt + champ produit + mouvements non réconciliés	Réception sans effet, incohérences	Stock unique, mouvements transactionnels	P0
Traçabilité	2 actions auditées, logs de démo	Pas de preuve	Audit systématique immuable	P0
Ventes	POS détail/gros, crédit	Prix client, ni annulation ni retour, ni TVA ni remise	Pièces à états, prix serveur, TVA, remises, avoirs	P0/P1
Achats	Commande → réception totale → facture → paiement	Réception KO, pas de partiel ni d'annulation	Réceptions partielles, CUMP, rapprochement	P1
Inventaire	Saisie globale appliquée immédiatement	Pertes masquées, pas de validation	Workflow ouverture/comptage/validation	P1
Caisse	Session unique globale	Pas de responsabilité par poste	Session par poste, opérateur et dépôt	P1
Catalogue	Produit simple, catégories figées	Ni marque, ni variantes, ni code-barres	Catalogue optique complet	P1/P2
Reporting	KPI du jour, rapports côté client, courbe fictive	Indicateurs partiels ou trompeurs	Rapports serveur fiables, valorisation	P2
Alertes	Libellé stock faible	Aucune anticipation	Alertes planifiées et notifications	P2
Lots / péremption	Absent	Rappels et périmés non gérés	Lots + FEFO (lentilles, solutions)	P2
UX	Material cohérent, desktop	Pas de mobile, pas de scan, listes non paginées	Responsive, scan, pagination serveur	P1/P2
Tests	2 tests (1 KO)	Aucune protection contre les régressions	Pyramide de tests + CI	P0/P1
Multi-société	Absent	—	<code>companyId</code> prévu dès le schéma	P3

20. Score de maturité

Méthode. Chaque domaine est noté de 0 à 100 selon une grille simple : 0–20 absent ou non fonctionnel ; 21–40 ébauche avec défauts bloquants ; 41–60 fonctionnel partiel ; 61–80 solide avec manques ; 81–100 niveau production. La note globale est la **moyenne pondérée** ci-dessous. Les pondérations reflètent l'importance pour un outil de gestion manipulant stock et argent : la sécurité et le fonctionnel pèsent plus lourd. Le score mesure la **maturité pour un usage en production**, pas la qualité de la maquette.

Domaine	Poids	Score /100	Pondéré	Commentaire
Fonctionnel	20 %	45	9,0	Large couverture d'écrans ; bug de réception, ni retours ni annulations, inventaire fragile
Architecture	10 %	40	4,0	Front bien structuré ; backend absent, logique dans le navigateur
Base de données	10 %	15	1,5	Aucune base ; modèles incomplets (dépôt manquant) ; schémas non branchés
API	10 %	30	3,0	Contrat cohérent ; pas de pagination, prix client, endpoints manquants ou erronés
UX/UI	10 %	55	5,5	Interface soignée desktop ; pas de mobile, de scan ni de menu par rôle
Sécurité	15 %	10	1,5	Aucune route protégée, tout le monde admin, mot de passe unique
Performance	5 %	35	1,75	Lazy loading OK ; tout chargé et agrégé côté client
Tests	5 %	3	0,15	1 test utile, 1 en échec
Maintenabilité	5 %	45	2,25	Conventions et structure saines ; doublons, <code>unknown/any</code> , code mort
Traçabilité	10 %	25	2,5	Modèle d'audit bon, alimentation quasi nulle
Total	100 %	—	31,15	Maturité actuelle : 31 / 100

En tant que **prototype fonctionnel front-end**, l'application se situerait autour de 65/100 : le travail d'interface et de modélisation est réel et réutilisable.

21. Registre des risques

#	Risque	Gravité	Probabilité	Impact	Recommandation
1	Mise en service en l'état (données perdues au rechargement)	Critique	Élevée si déployé	Perte totale de ventes et de stock	Ne pas déployer avant le backend (phase 1)
2	Accès non authentifié à toutes les fonctions	Critique	Certaine	Fraude, fuite de données	Guards + auth serveur (phase 0/1)
3	Stock faux après réceptions	Critique	Certaine	Ventes refusées à tort, ruptures non vues, achats inutiles	Modèle de stock unique (phase 1)
4	Falsification des prix et marges	Élevée	Moyenne	Pertes financières non détectées	Prix serveur (phase 1)
5	Ajustements de stock non tracés (fiche produit)	Élevée	Élevée	Démarque inconnue, vols masqués	Ajustement motivé et audité
6	Inventaire pré-rempli masquant les écarts	Élevée	Élevée	Pertes invisibles	Workflow d'inventaire
7	Vulnérabilités de dépendances (Angular, xlsx)	Élevée	Moyenne	XSS, déni de service	Mises à jour (phase 0)
8	Caisse globale sans responsabilité individuelle	Élevée	Élevée (multi-postes)	Écarts non attribuables	Session par poste et opérateur
9	Absence de tests lors de la construction du backend	Élevée	Élevée	Régressions sur stock et argent	Tests dès la phase 1
10	Dérive de périmètre vers la comptabilité complète	Moyenne	Moyenne	Retard, complexité	S'en tenir à la gestion commerciale + export
11	Choix de SGBD non tranché (Mongo vs SQL dans la doc)	Moyenne	Moyenne	Perte de temps	Décider PostgreSQL en phase 0
12	Perte de connaissance (mono-développeur, doc générique)	Moyenne	Moyenne	Maintenance difficile	README réel, ADR, OpenAPI
13	Produits périmés vendus (lentilles, solutions)	Moyenne	Moyenne	Santé du client, image	Lots + FEFO (phase 3)
14	Dates en UTC dans les rapports	Faible (GMT)	Faible	Ventes de minuit mal datées	Fuseau configuré côté serveur

22. Roadmap

Estimations en semaines-développeur (1 développeur full-stack confirmé), indicatives.

Phase 0 — Stabilisation (1 à 2 semaines)

- **Objectifs** : sécuriser la base front et préparer le backend.
- **Fonctionnalités** :
 - corriger l'ordre des intercepteurs et supprimer le fallback admin ;
 - brancher les guards ;
 - unifier `AuthService` et `User` ;
 - mettre à jour Angular et remplacer `xlsx` ;
 - retirer la courbe de stock fictive ;
 - rendre le stock non éditable dans la fiche produit ;
 - corriger le test cassé, mettre en place la CI (lint, build, test) ;
 - acter les choix (NestJS, PostgreSQL) en ADR.
- **Dépendances** : aucune. **Complexité** : faible.
- **Risques** : faibles.
- **Résultat attendu** : un prototype honnête et sécurisable, une base saine.

Phase 1 — Fondations (6 à 8 semaines)

- **Objectifs** : backend, persistance, stock fiable, utilisateurs.
- **Fonctionnalités** :
 - API NestJS + PostgreSQL + migrations ;
 - auth réelle et RBAC ;
 - catalogue (produits, catégories, marques, dépôts) ;
 - modèle de stock unique avec mouvements transactionnels et `warehouseId` partout ;
 - audit systématique ;
 - pagination serveur ;
 - branchement des facades existantes sur l'API réelle ;
 - tests unitaires et d'intégration du domaine stock.
- **Dépendances** : phase 0. **Complexité** : élevée.
- **Risques** : sous-estimation de la migration des règles de `MBI` .
- **Résultat attendu** : **V1 interne utilisable** pour le catalogue et le stock.

Phase 2 — Gestion commerciale (5 à 7 semaines)

- **Objectifs** : ventes, achats, clients, fournisseurs, caisse en conditions réelles.
- **Fonctionnalités** :
 - ventes avec prix serveur, TVA, remises, numérotation, annulation, retours et avoirs ;
 - achats avec réceptions partielles, annulation et CUMP ;
 - créances et dettes ;
 - caisse par poste et opérateur ;
 - factures PDF côté serveur.
- **Dépendances** : phase 1. **Complexité** : élevée.
- **Risques** : règles fiscales locales à valider avec le comptable.
- **Résultat attendu** : **V1 exploitable en boutique**.

Phase 3 — Stock avancé (4 à 6 semaines)

- **Objectifs** : fiabilité physique du stock.
- **Fonctionnalités** :

- inventaire en workflow ;
- transferts en workflow ;
- code-barres (scan POS et inventaire, étiquettes) ;
- lots et péremption sur les catégories concernées, FEFO.
- **Dépendances** : phases 1 et 2. **Complexité** : moyenne à élevée.
- **Résultat attendu** : écarts d'inventaire mesurés et justifiés.

Phase 4 — Reporting (3 à 4 semaines)

- **Objectifs** : pilotage fiable.
- **Fonctionnalités** :
 - dashboard serveur (valeur du stock, ruptures, sous seuil, CA, marge, créances, dettes) ;
 - rapports : valorisation, rotation, dormants, top ventes, achats, inventaires, performance par dépôt et par vendeur ;
 - exports ;
 - export comptable.
- **Dépendances** : phases 1 à 3. **Complexité** : moyenne.
- **Résultat attendu** : décisions basées sur des chiffres exacts.

Phase 5 — Optimisation (3 à 4 semaines)

- **Objectifs** : performance, UX et mobilité.
- **Fonctionnalités** :
 - responsive et tablette ;
 - raccourcis clavier au POS ;
 - vues matérialisées ;
 - alertes planifiées ;
 - PWA (lecture hors-ligne) ;
 - scan caméra ;
 - E2E Playwright complets.
- **Dépendances** : phases 1 à 4. **Complexité** : moyenne.

Phase 6 — Fonctionnalités avancées (selon pertinence)

- Dossier patient et ordonnances (optique) — **recommandé en priorité dans cette phase**.
- Notifications SMS/WhatsApp, réapprovisionnement automatique, Mobile Money, API externe, multi-entreprise (si projet SaaS).



23. Priorisation Impact × Effort

Quick wins — fort impact, faible effort

- Ordre des intercepteurs + suppression du fallback admin
- Brancher les guards existants
- Stock non éditable dans la fiche produit
- Retirer la courbe de stock fictive ; libellé « marge brute »
- Contrôle `isActive` au login
- Mise à jour Angular, remplacement de `xlsx`
- Bloquer le crédit sans client côté API, le surpaiement client
- Menu filtré par rôle
- Corriger le test cassé + CI

Projets prioritaires — fort impact, effort moyen à élevé

- Backend + PostgreSQL + auth réelle
- Modèle de stock unique et transactionnel
- Prix et coûts serveur, CUMP
- Audit systématique
- Annulation, retours, avoirs
- Réceptions partielles
- Workflow d'inventaire
- Caisse par poste
- Pagination serveur

Améliorations secondaires — impact moyen

- Code-barres et étiquettes
- Alertes planifiées
- Rapports avancés, export comptable
- Transferts en workflow
- Variantes produit, marques
- Responsive tablette
- Lots et péremption (lentilles, solutions)

À reporter — faible impact ou effort élevé

- Multi-entreprise (prévoir seulement la colonne)
- Emplacements fins (zones, rayons)
- Mode hors-ligne complet
- Réapprovisionnement automatique
- API publique externe

24. Conclusion

1. **Niveau réel de maturité : 31/100.** C'est un prototype fonctionnel front-end avancé (environ 65/100 dans cette catégorie), sans backend ni persistance. Il n'est pas utilisable en production.
2. **Points forts :**
 - périmètre fonctionnel large et pertinent pour un opticien ;
 - front Angular moderne, bien structuré et homogène ;
 - règles métier déjà formalisées dans le mock (crédit, caisse, paiements partiels) ;
 - contrat d'API esquissé ;
 - modèle d'audit bien pensé.
3. **Principaux problèmes :**
 - absence de backend et de base de données ;
 - sécurité inopérante (routes ouvertes, tout le monde admin) ;
 - stock incohérent (réception sans effet, double source, mouvements sans dépôt) ;
 - prix falsifiables ;
 - traçabilité de façade ;
 - ni annulation ni retours ;
 - tests inexistants.
4. **Fonctionnalités indispensables :** backend persistant, authentification et RBAC, stock unique transactionnel par dépôt, prix serveur, audit, annulation et retours, réceptions partielles, inventaire validé, caisse par poste.
5. **Risques à corriger immédiatement :** routes non protégées, fallback admin et ordre des intercepteurs, édition directe du stock, dépendances vulnérables, indicateur fictif. Et surtout : **ne pas mettre l'application en service** dans son état actuel.
6. **Architecture cible :** front Angular conservé, API NestJS modulaire, PostgreSQL, JWT et refresh HttpOnly, RBAC à permissions, audit transactionnel, jobs planifiés, stockage objet.
7. **Roadmap :** phase 0 (stabilisation, 1–2 sem.) → phase 1 (fondations, 6–8 sem.) → phase 2 (gestion commerciale, 5–7 sem.) = **V1 boutique en ~12–17 semaines** ; puis stock avancé, reporting, optimisation, fonctions avancées.
8. **Évolution ou refonte ? Évolution** pour le front. Pour le backend, il s'agit d'une **création**, pas d'une refonte : **MBI** sert de spécification et sera conservé comme mode démo. Seule une **refonte partielle du modèle de données** (stock, rattachement au dépôt, états des pièces) est nécessaire.
9. **Prochaine étape concrète :** valider cette feuille de route et la stack, puis lancer la phase 0. Premier lot de 2 semaines : corrections de sécurité front + squelette NestJS/PostgreSQL avec le module `auth` et le module `inventory` (stock par dépôt, `applyMovement` transactionnel, tests), branché sur l'écran Produits/Stock existant.

25. Annexes

25.1 Méthode et vérifications effectuées

- Lecture de l'intégralité des modèles, services, intercepteurs, guards, routes, du backend simulé et des facades et pages principales.
- `ng build` (production, sortie hors projet) : **succès**, avertissements CommonJS (jspdf/canvg/html2canvas).
- `ng test --watch=false` : **2 tests, 1 échec** (`app.spec.ts:21`).
- `npm audit --omit=dev` : **10 vulnérabilités** (5 high, 5 moderate).
- Inspection du bundle de production : présence des données mock.
- `git status` en fin d'audit : **arbre de travail propre**, aucun fichier du projet modifié.

25.2 Points À CONFIRMER

Point	Raison
Comportement d'une future API réelle	Inexistante ; seul le contrat simulé est vérifiable
Exigences fiscales locales (TVA, mentions de facture, numérotation)	Hors code ; à valider avec un expert-comptable (Côte d'Ivoire, d'après les données)
Volumétrie réelle attendue (références, ventes/jour, postes)	Non documentée ; les recommandations de performance supposent 1 000 à 10 000 références
Besoin réel du module Rendez-vous et d'un dossier patient	Domaine optique déduit des données de démonstration
Usage visé des fichiers <code>seeds/*.sql js</code>	Présents dans <code>src/</code> mais non exécutés par l'application

25.3 Index des preuves principales

Constat	Fichier : ligne
Réception sans mise à jour du stock dépôt	MBI:1219-1229
Produits renvoyés avec le stock du dépôt	MBI:530
Fallback admin sans token	MBI:377-382
Ordre des intercepteurs	<code>src/app/app.config.ts:21</code>
Routes sans guard	<code>src/app/routing.ts</code> , <code>src/app/layout/layout.routes.ts</code>
Mot de passe universel	MBI:485 ; <code>login.component.html:129</code>
Prix de vente fournis par le client	MBI:1663-1676
Stock modifiable via PUT produit	MBI:692-695 ; <code>product-drawer.component.ts</code> (champ <code>stockQuantity</code>)
Mouvement manuel sans contrôle de rôle	MBI:1610-1648
Audit : 2 appels réels	MBI:701, 1741
Rapports API non filtrés	MBI:1862-1970
Évolution du stock fictive	<code>features/dashboard/data/dashboard.facade.ts:152-165</code>
Inventaire pré-rempli	<code>features/inventory/pages/inventory-count-page.component.ts:58-63</code>
Inventaire appliqué immédiatement	MBI:789-805
Login sans contrôle <code>isActive</code>	MBI:482-489
Deux modèles de rôles	<code>core/models/role.model.ts</code> , <code>auth/models/user.model.ts</code>
PATCH statut RDV : mauvais index	MBI:1768